Evidence of successful completion of the activities and learning.

Exercise 2.2.1

Client-server architecture:

- A web application/server uses a client-server model.
- The client sends an HTTP request.
- The server will listen on a TCP socket, process the received request through the application layer and will return an HTTP response.
- TLS is used to ensure confidentiality, integrity and server authentication when HTTPS is used.
- The web server might also communicate with other services like APIs, database servers, authentication providers, etc...
- This architecture allows the administrators to have centralized logging, data stores and app states.

Peer to peer architecture:

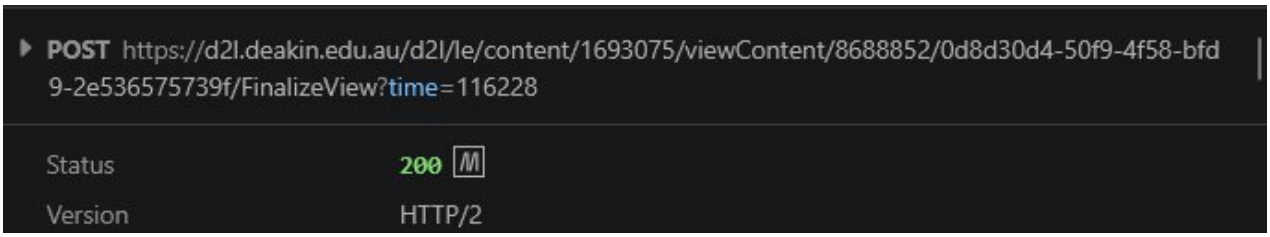
- BitTorrent is a decentralized p2p file distribution protocol where each of its "nodes" (devices in the network/swarm) can operate either as a client or a server.
- Each file being shared is fixed into small pieces (chunks) allowing peers to download them simultaneously from multiple services and recreate the file at the every end.
- Because of this architecture, it allows nodes even without the full file to still share the chunks they have.
- When exchanging these files, peers use rarest file selection algorithms and a lot of other techniques to select which chunks to download the first.
- Trackers with distributed hash tables are used to discover peers, while the actual transfer happens between the peers directly.
- The more devices in the swarm, the more reliable and fast the file distribution will become.

Exercise 2.2.2

I use `ping` everyday for basic troubleshooting. It uses the ICMP protocol - the ICMP Echo Request (type 8) and ICMP Echo Reply (type 0) to be specific. And, ICMP is a network layer protocol. Therefore, this cannot be included here[1].

In general day to day web browsing, we use HTTP and HTTPS. This traffic usually uses TCP but HTTP3 uses QUIC over UDP[2]. For emails, application layer protocols like SMTP and IMAP are used with TCP as the transport layer protocol. For DNS lookups, we usually use UDP (53/udp). Sometimes, there might also be a TCP fallback. SIP (Session Initiation Protocol) is mainly used in VoIP and calls and it usually uses UDP, however, we can also configure it to use TCP if required.

Exercise 2.4.1



My browser uses persistent HTTP connections. Instead of creating a new connection for every object, it simply reuses an already established connection when having to deal with multiple HTTP requests and responses.

HTTP 1.1 uses persistent connections by default. HTTP 2 can multiple multiple request-response streams over one TCP connection and HTTP 3 also provides a similar multiplexing capability over QUIC. Therefore, modern browser traffic is generally persistent unless the server explicitly closes the connection or doesn't support connection reuse.

Multiplexing allows several HTTP requests to be sent over the same connection at the same time and their responses can arrive in any interleaved order. This is used by both HTTP 2 and 3 to ensure that one slow response doesn't completely block everything else.

Exercise 2.4.2

- GET: retrieves a resource
- POST: sends data to the server for processing. Very commonly used for form submissions, authentication, file uploads, and similar actions.
- PUT: create or fully replace an existing resource
- PATCH: update a resource without replacing everything
- DELETE: remove a resource
- HEAD: works like GET but will only return the response headers and not the response body.
- OPTIONS: request information about methods and features supported by a server (or a resource)
- QUERY: perform a read only server side query with the query parameters in the request body. introduced in June 2026 in RFC 10008[3]

Exercise 2.4.3

Command to run is: `curl -sI -L <URL>` – based on this stack overflow answer[4].

```
PS C:\Users\uncus> curl.exe -sI -L https://www.google.com
HTTP/1.1 200 OK
Content-Type: text/html; charset=ISO-8859-1
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-GjCWoRN6gH9Pi_XMCdwOvA' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
Accept-CH: Sec-CH-Prefers-Color-Scheme
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Date: Sun, 26 Jul 2026 11:22:43 GMT
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Expires: Sun, 26 Jul 2026 11:22:43 GMT
Cache-Control: private
Set-Cookie: __Secure-STRP=ANmZwa1KyQ6oWIb6zp7seo3v6tNazB1FQQ53vMoTF4PZqx1LmdsuXPFDP3uzdVsRbDDAMI6PtVXOr277d66TI23gkqsCZGsu8aLM; expires=Sun, 26-Jul-2026 11:27:43 GMT; path=/; domain=.google.com; Secure; SameSite=strict
Set-Cookie: AEC=AdJVEavjXx0BeORvZIBTmkXPpZ3yDGSybI9Rt_CmPMTF4Qfw4pTj6_kyIg; expires=Fri, 22-Jan-2027 11:22:43 GMT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax
Set-Cookie: NID=533=m_3lHuxKOaigVVHYeOasG1BzekNZNPI1uAEDbggTpSwAFib9sV3kT3eKGaKKBfjq1th3zwnzx0Sf0ZjODJseVHmEzoRKm6hGw5X6FltrDuc-IMpcvtIi5eqPu9M0IRZequniHmWa94LuS-u5WV173Pgo-RivFgSeZ5AvzPkmnOg1f_IO63rYw6__ow_yuQNoVwAetQuadwTSIdAD4Q; expires=Mon, 25-Jan-2027 11:22:43 GMT; path=/; domain=.google.com; HttpOnly
Transfer-Encoding: chunked
Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

This command displays all the headers. Since we want to filter out cookie details, I will just grep it / use findstr.

Three cookies set from google.com. There are three cookies.

- __Secure-STRP
  - has not been documented officially but seems to be something security related and expires within 5 minutes.
  - Response was generated at 11:26:45 GMT and expires at 11:31:45 GMT.
  - Cookie will only be sent during same site interactions. It won't be sent when user visits google from another site to protect against XSS attacks.
  - Might be accessible via document.cookie.
- AEC
  - Used to detect spam, fraud and abuse by google.
  - Expires in 22 January 2027, 11:26:45 GMT
  - Secure: Only sent through HTTPS
  - HttpOnly: not accessible via document.cookie
  - SameSite=Lax: generally blocked from cross site subrequests. However, it might still be available if the user directly goes to google through a normal top level link.
  - Path=/: applies to every URL path under the specified domain.
- NID
  - Used to store user preferences and to support with analytics and advertising.
  - It's HttpOnly.
  - However, it doesn't have the Secure attribute set. This means the cookie is not restircted to HTTPS - but google uses HTTPS and HSTS.
  - Has no SameSite attribute explicitly set, but by default, chromium treats this as SameSite=Lax[5].

Functionalities of these cookies can be found here[6]

```
PS C:\Users\uncus> curl.exe -sI -L https://www.google.com | findstr /I "Set-Cookie"
Set-Cookie: __Secure-STRP=ANmZwa3JA26o_kiWdq6xunoQw3y648dCtciIqkNPKzltNdZP9DBOCQY_wsd8QKhASafAeGIcMjlhlaFzLGNiTcf235ZRfw
ZIlkGk; expires=Sun, 26-Jul-2026 11:31:45 GMT; path=/; domain=.google.com; Secure; SameSite=strict
Set-Cookie: AEC=AdJVEauKPYDp5NR1iTuOAOS7NSBU1iESsH7aVeeBF-9Qt71X1vE2xjJ4AQ; expires=Fri, 22-Jan-2027 11:26:45 GMT; path=
/; domain=.google.com; Secure; HttpOnly; SameSite=lax
Set-Cookie: NID=533=oqO_lY2aRP45ODZCctT2OyzcLbIaCkB3s8VvFkB5adnSo9ySM9y_x9rIWkK93dV9k9mxpqKZZaxUr1Sy483N9S1XSbg_DYJsge2m
Qro3GC_sUpYO2y0Xf1NG8lxKM3Rulq_yd7Tp3KNG8La72Oo_lNKpIwi95H4xVI79VVqT8f_xwHTIOY7347rvayjUjrLypYbGjZ2rqkNV-TVOkQ; expires=
Mon, 25-Jan-2027 11:26:45 GMT; path=/; domain=.google.com; HttpOnly
PS C:\Users\uncus>
```

However, with amazon, at first, the response was a "503 Service Temporarily Unavailable".

```
PS C:\Users\uncus> curl.exe -sI -L https://www.amazon.com.au
HTTP/1.1 503 Service Temporarily Unavailable
Content-Type: application/octet-stream
Content-Length: 1249
Connection: keep-alive
Server: Server
Date: Sun, 26 Jul 2026 11:27:34 GMT
ETag: "6a367fa8-4e1"
x-amz-rid: 2C6TKBJMA5ZZ038J9VJW
Strict-Transport-Security: max-age=47474747; includeSubDomains; preload
X-Frame-Options: SAMEORIGIN
Edge-Cache-Control: no-store,no-cache,stale-if-error=0,stale-while-revalidate=0
Vary: User-Agent,Accept-Encoding
X-Cache: Error from cloudfront
Via: 1.1 72f9ca7159c18e5ed6c60c63d630a784.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MEL52-P1
Alt-Svc: h3=":443"; ma=86400
X-Amz-Cf-Id: m60dymd3Ev37a8qGH5Whzi6SkIWIzfVnF-TgftYABlz2sW1CzYg23w==
```

Next, I attempted it with a user agent and I got a "405 Method Not Allowed". The only allowed methods here are "GET, POST, PUT, DELETE, OPTIONS" as seen below.

```
PS C:\Users\uncus> curl.exe -sI -L -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/138.0.0.0 Safari/537.3" https://www.amazon.com.au
HTTP/1.1 405 Method Not Allowed
Content-Type: text/html;charset=UTF-8
Content-Length: 0
Connection: keep-alive
Server: Server
Date: Sun, 26 Jul 2026 11:29:59 GMT
Edge-Cache-Control: no-store,no-cache,stale-if-error=0,stale-while-revalidate=0
x-amz-rid: 2G1QZK5AZMQ7X6W3DAQ2
Cache-Control: no-cache
Pragma: no-cache
Expires: -1
Allow: GET, POST, PUT, DELETE, OPTIONS
Vary: Accept-Encoding,User-Agent,Accept-Encoding,User-Agent
Strict-Transport-Security: max-age=47474747; includeSubDomains; preload
X-Frame-Options: SAMEORIGIN
X-Cache: Error from cloudfront
Via: 1.1 7c189e89cce55d20687c20fcdaf9bacc.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MEL52-P1
Alt-Svc: h3=":443"; ma=86400
X-Amz-Cf-Id: vFCfmXWSP6Yqo-uxeGVcCUjCWMhbze8S3TG3UY4AGxlkomGaCFwTnw==
```

Exercise 2.6.1

```
Wireless LAN adapter WiFi:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
   Physical Address. . . . . . . . . : F4-A4-75-A2-46-B2
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::5766:b73e:270a:41b5%17(Preferred)
   IPv4 Address. . . . . . . . . . . : 10.54.254.167(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Sunday, 26 July 2026 8:16:11 PM
   Lease Expires . . . . . . . . . . : Sunday, 26 July 2026 10:16:28 PM
   Default Gateway . . . . . . . . . : 10.54.254.50
   DHCP Server . . . . . . . . . . . : 10.54.254.50
   DHCPv6 IAID . . . . . . . . . . . : 183805045
   DHCPv6 Client DUID. . . . . . . . : 00-01-01-00-31-F1-F4-DA-88-A4-C2-3B-43-99
   DNS Servers . . . . . . . . . . . : 10.54.254.50
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

Exercise 2.6.2

- User enters www.nasa.gov in their browser.
- Browser auto converts it to `https://www.nasa.gov`
- Browser checks the cache, cookies, HSTS and existing connections.
- DNS resolves www.nasa.gov to an IPv4/v6 address.
- Client uses ARP (IPv4) / NDP (Neighbor Discovery Protocoll for IPv6) to find the local routers MAC address
- The packet is sent through the router, ISP and internet to NASA's CDN/server.
- The router may perform NAT and firewall checks.
- The client then starts the TCP connection
    - TCP three way handshake happens for HTTP 1.1 and 2.
    - QUIC over UDP is used for HTTP 3.

- The TLS handshake occurs.
    - Browser sends `ClientHello`
    - Server sends certificate and cryptographic information.
    - The browser verifies this certificate.
    - Encryption keys will be created.
- Browser will send an encrypted HTTP `GET /` request.
- NASA's CDN/server processes the request.
- Server sends the HTML response back.
- Browser decrypts this and decompresses this response
- Browser parses the HTML
- Browser requests extra files (css, js, images, fonts, etc...)
- Additional HTTP requests may get sent to other domains following the same process too.
- Browser builds the page layout / DOM and renders it on the screen.

Workshop Activity 2.2

1. Why do we need application layer protocols?

- It defines how applications communicate over a network.
- They specify the message format, syntax, semantics and timing.
- They enable interoperability between clients and servers.

2. Discuss some examples of application layer protocols.

- HTTP(S): web communication
- DNS: domain name to ip resolution
- SMTP: send emails
- IMAP/POP3: receive emails
- FTP/SFTP: file transfer
- DHCP: host configuration
- SSH: secure remote access

3a. HTTP is a client-server protocol. Can you identify the key features of HTTP?

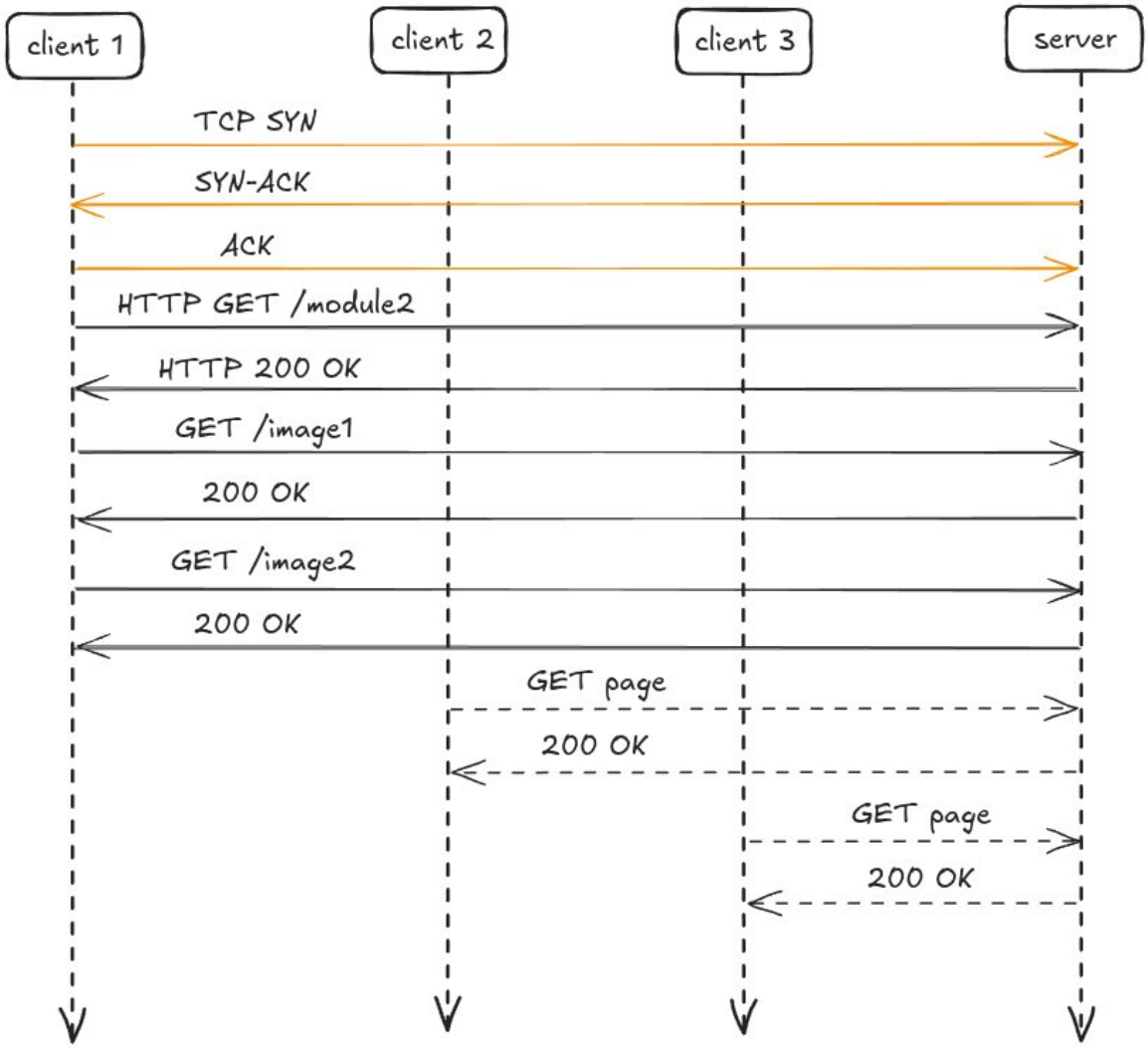
- It's a client server and request-response protocol.
- The client is usually a web browser and it sends a request to the web server and the server returns a response.
- HTTP is a stateless protocol. The server does not automatically remember the precious requests from the same client.
- As a workaround for this, we use sessions and cookies to maintain the state.
- It supports multiple methods like GET, POST, DELETE to do various sorts of things.

3b. Identify a group member who could act as a web server and other three members can act as clients. So, your network has one server and three clients.

- For this, a group needs 5 people.
- I'm doing this by myself. Therefore, I will just visualize this.

3c. Let’s assume each client wants to access a web page stored in the web server and we use HTTP to communicate in the application layer level. You can now act as a server-client model communicating the right messages between each entity in sequence to get the information that each client wants. For example, Client 1 could say: "Client 1 initiating TCP connection with the Server". Then, the server replies with the correct response. You need to note down the correct messages in sequence.

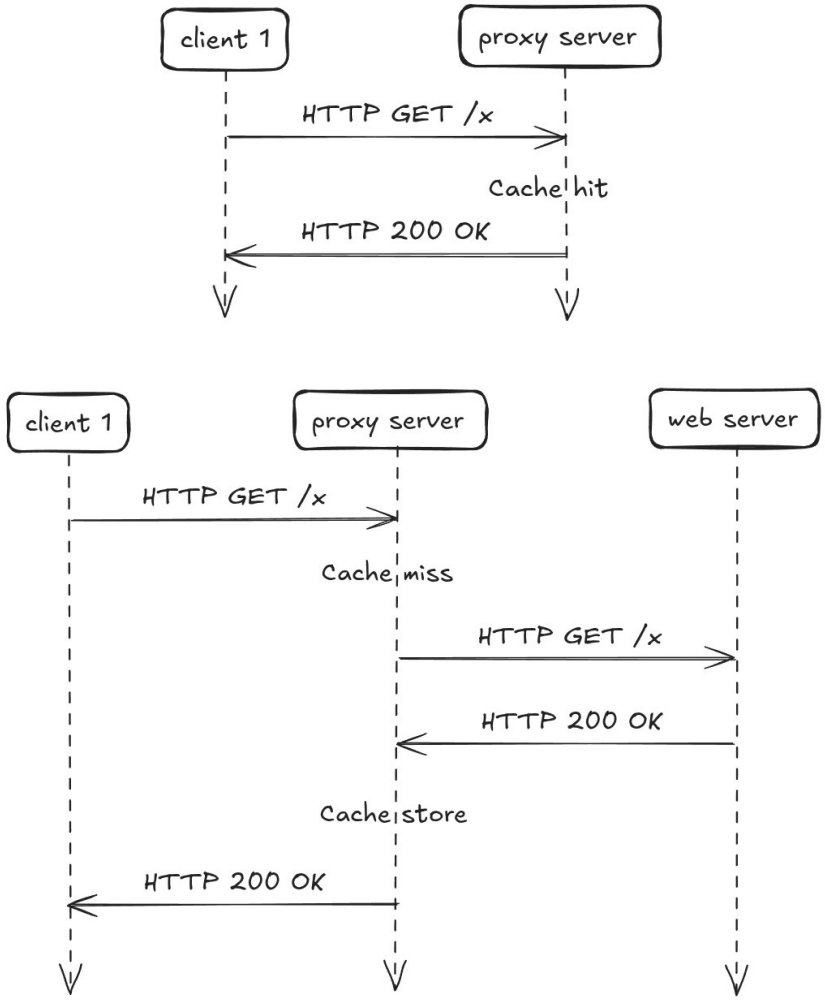
- The client resolves the web servers' IP using DNS
- The client starts a TCP connection with the server by sending a `SYN`
- The server replies with a `SYN-ACK`
- The client sends back `ACK` to complete the TCP three way handshake.
- If HTTPS is being used, the client and the server will then complete the TLS handshake.
- Once all the initial setup is done, the client will send an HTTP request like `GET /module2 HTTP/1.1`.
- After that, the server will respond with something like `HTTP/1.1 200 OK`
- The response will contain the HTTP web page, and based on the links here, the client browser might send more GET requests (sometimes from other domains, doing this process all over again) to fetch other CSS, JS, images, etc...



A proxy server / web cache sits between the real client and the web server. Instead of the client directly requesting resources from the web server, the request will first go through the proxy. This way, if it has something that it requested before, it can reuse them now. THis will help reduce the load to the web server, the loading times, bandwidth usage, and many other system resources. Proxy servers can also be used for logging and access control.

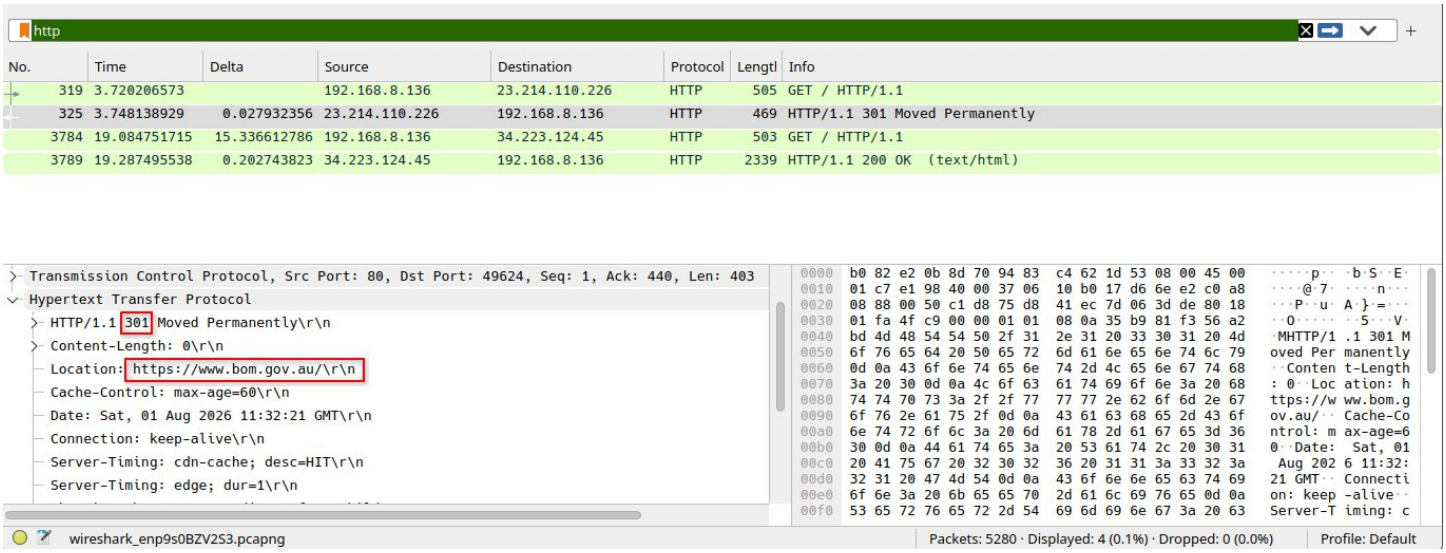
I have personal experience setting up Squid cache for a Discourse instance, and when done right, it reduced the requests sent to the web server by around 40 percent.

When a client requests and object, the proxy server will first check whether it already has a valid copy stored in its cache. If its available, its called a cache hit and the proxy server will return this object directly to the client. If its not available, its called a cache miss and the proxy will forward this request to the web server, will receive the object and and then send it back to the client, and it will also store it for future requests it might receive (based on the HTTP caching headers).



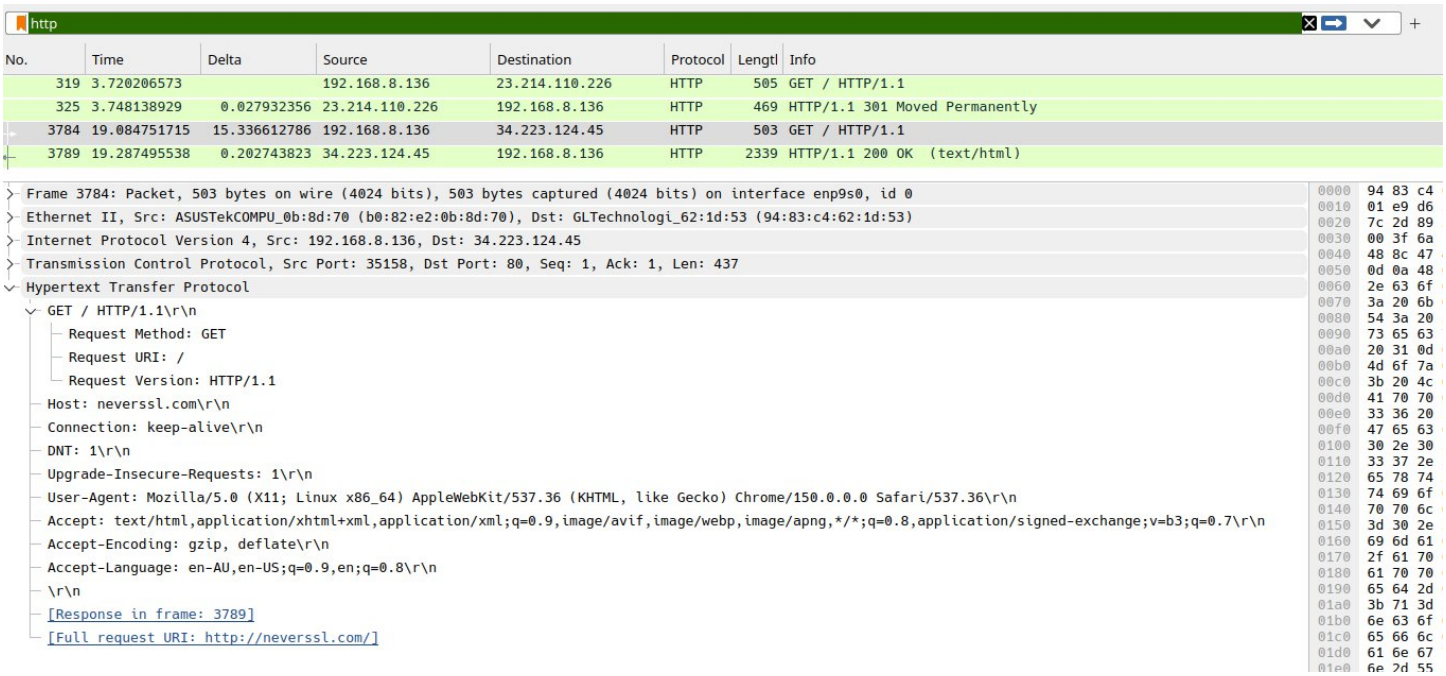
Workshop Activity 2.3

Visiting http://www.bom.gov.au/ redirects you (301 Permanent Redirect) to the https version of that. As it uses TLS, the response's contents won't be readable.



Therefore, I visited the popular http://neverssl.com/ website. The packet capture can downloaded here: https://drive.google.com/file/d/1H0wq0Hr-HlefHglHEn27XEQc0Dsppj4l/view?usp=sharing

In the screenshot below, you can clearly see the `GET` request, the Request Version, `Host`, `User-Agent` and the `Connection` type.



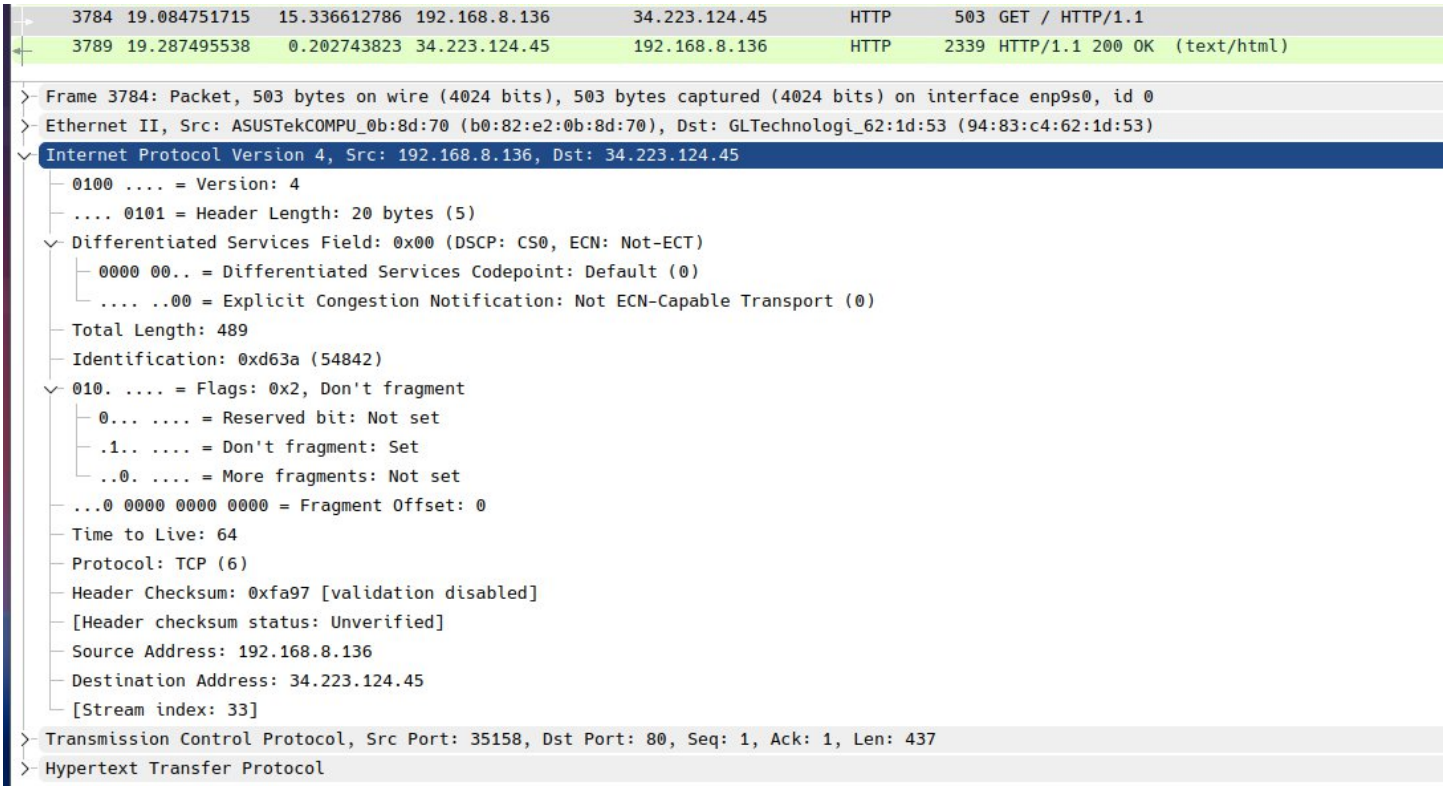
```
   3784 19.084751715   15.336612786 192.168.8.136        34.223.124.45        HTTP      503 GET / HTTP/1.1
   3789 19.287495538    0.202743823 34.223.124.45        192.168.8.136        HTTP     2339 HTTP/1.1 200 OK  (text/html)

> Frame 3789: Packet, 2339 bytes on wire (18712 bits), 2339 bytes captured (18712 bits) on interface enp9s0, id 0
> Ethernet II, Src: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53), Dst: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)
> Internet Protocol Version 4, Src: 34.223.124.45, Dst: 192.168.8.136
> Transmission Control Protocol, Src Port: 80, Dst Port: 35158, Seq: 1, Ack: 438, Len: 2273
v Hypertext Transfer Protocol
  > HTTP/1.1 200 OK\r\n
    Date: Sat, 01 Aug 2026 11:32:36 GMT\r\n
    Server: Apache/2.4.66 ()\r\n
    Upgrade: h2,h2c\r\n
    Connection: Upgrade, Keep-Alive\r\n
    Last-Modified: Wed, 29 Jun 2022 00:23:33 GMT\r\n
    ETag: "f79-5e28b29d38e93-gzip"\r\n
    Accept-Ranges: bytes\r\n
    Vary: Accept-Encoding\r\n
    Content-Encoding: gzip\r\n
  v Content-Length: 1900\r\n
      [Content length: 1900]
    Keep-Alive: timeout=5, max=100\r\n
    Content-Type: text/html; charset=UTF-8\r\n
    \r\n
    [Request in frame: 3784]
    [Time since request: 202.743823 milliseconds]
    [Request URI: /]
    [Full request URI: http://neverssl.com/]
    Content-encoded entity body (gzip): 1900 bytes -> 3961 bytes
    File Data: 3961 bytes
> Line-based text data: text/html (131 lines)
```

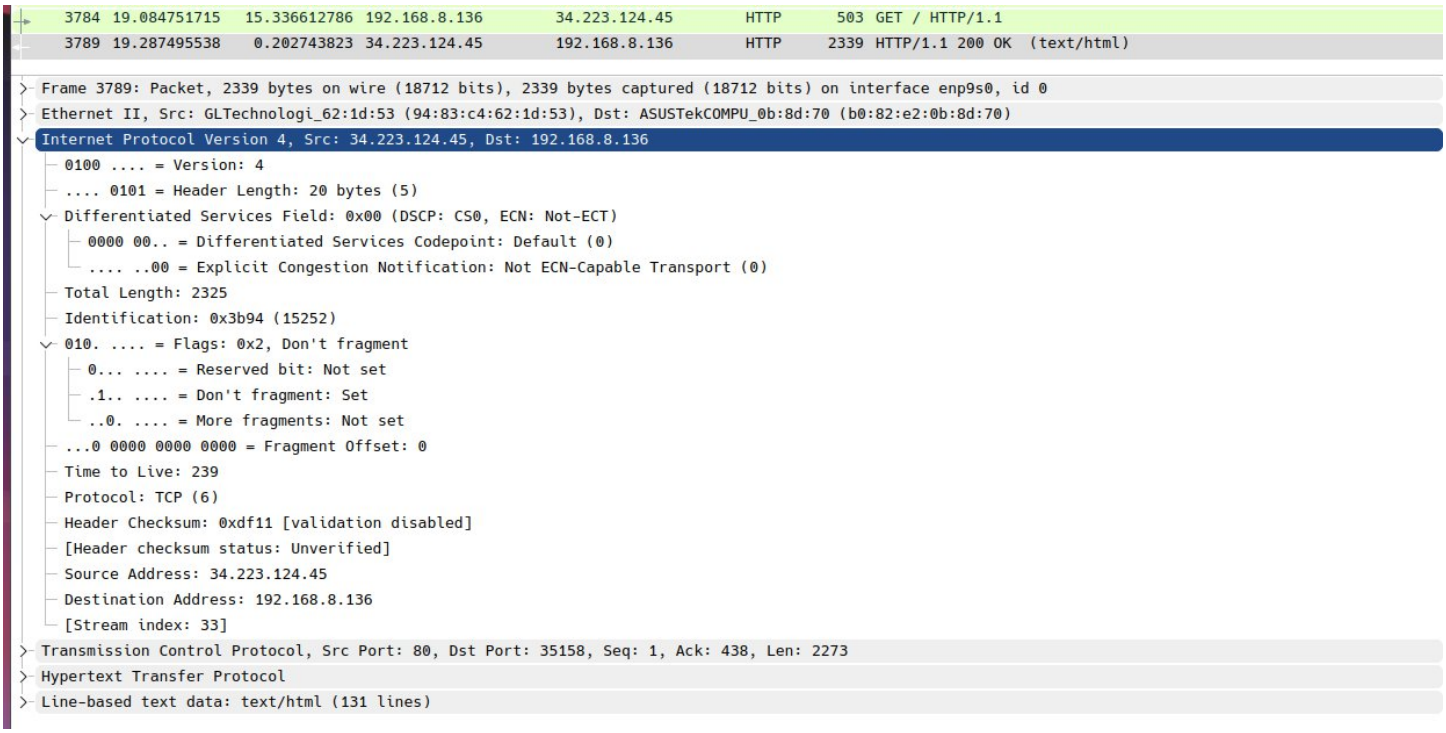
You can see the Ethernet II details.

```
   3784 19.084751715   15.336612786 192.168.8.136        34.223.124.45        HTTP      503 GET / HTTP/1.1
   3789 19.287495538    0.202743823 34.223.124.45        192.168.8.136        HTTP     2339 HTTP/1.1 200 OK  (text/html)

> Frame 3784: Packet, 503 bytes on wire (4024 bits), 503 bytes captured (4024 bits) on interface enp9s0, id 0
v Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
  v Destination: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  v Source: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 34.223.124.45
> Transmission Control Protocol, Src Port: 35158, Dst Port: 80, Seq: 1, Ack: 1, Len: 437
> Hypertext Transfer Protocol
```

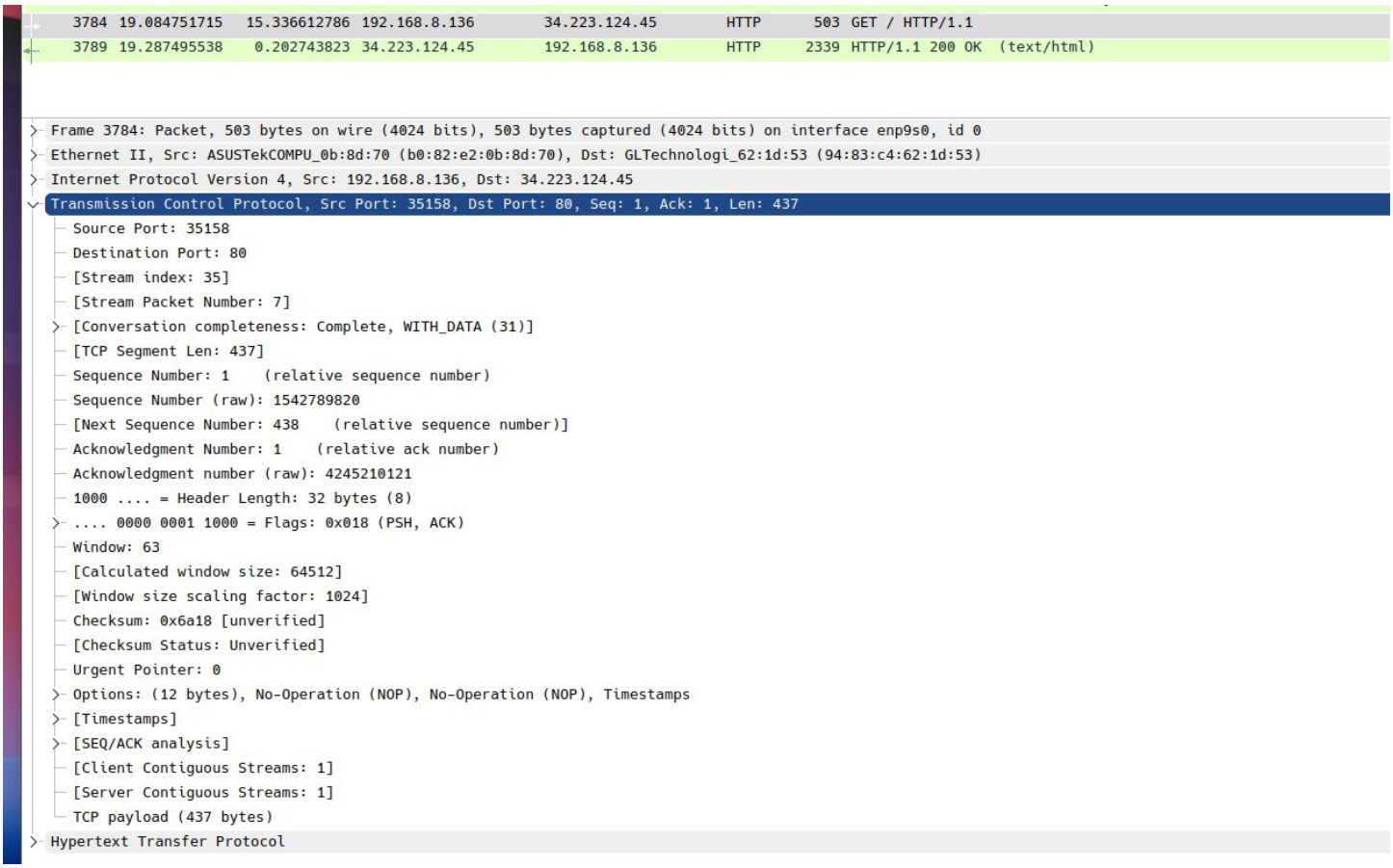
```
   3784 19.084751715   15.336612786 192.168.8.136        34.223.124.45        HTTP      503 GET / HTTP/1.1
   3789 19.287495538    0.202743823 34.223.124.45        192.168.8.136        HTTP     2339 HTTP/1.1 200 OK  (text/html)

> Frame 3789: Packet, 2339 bytes on wire (18712 bits), 2339 bytes captured (18712 bits) on interface enp9s0, id 0
v Ethernet II, Src: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53), Dst: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)
  v Destination: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  v Source: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
> Internet Protocol Version 4, Src: 34.223.124.45, Dst: 192.168.8.136
> Transmission Control Protocol, Src Port: 80, Dst Port: 35158, Seq: 1, Ack: 438, Len: 2273
> Hypertext Transfer Protocol
> Line-based text data: text/html (131 lines)
```

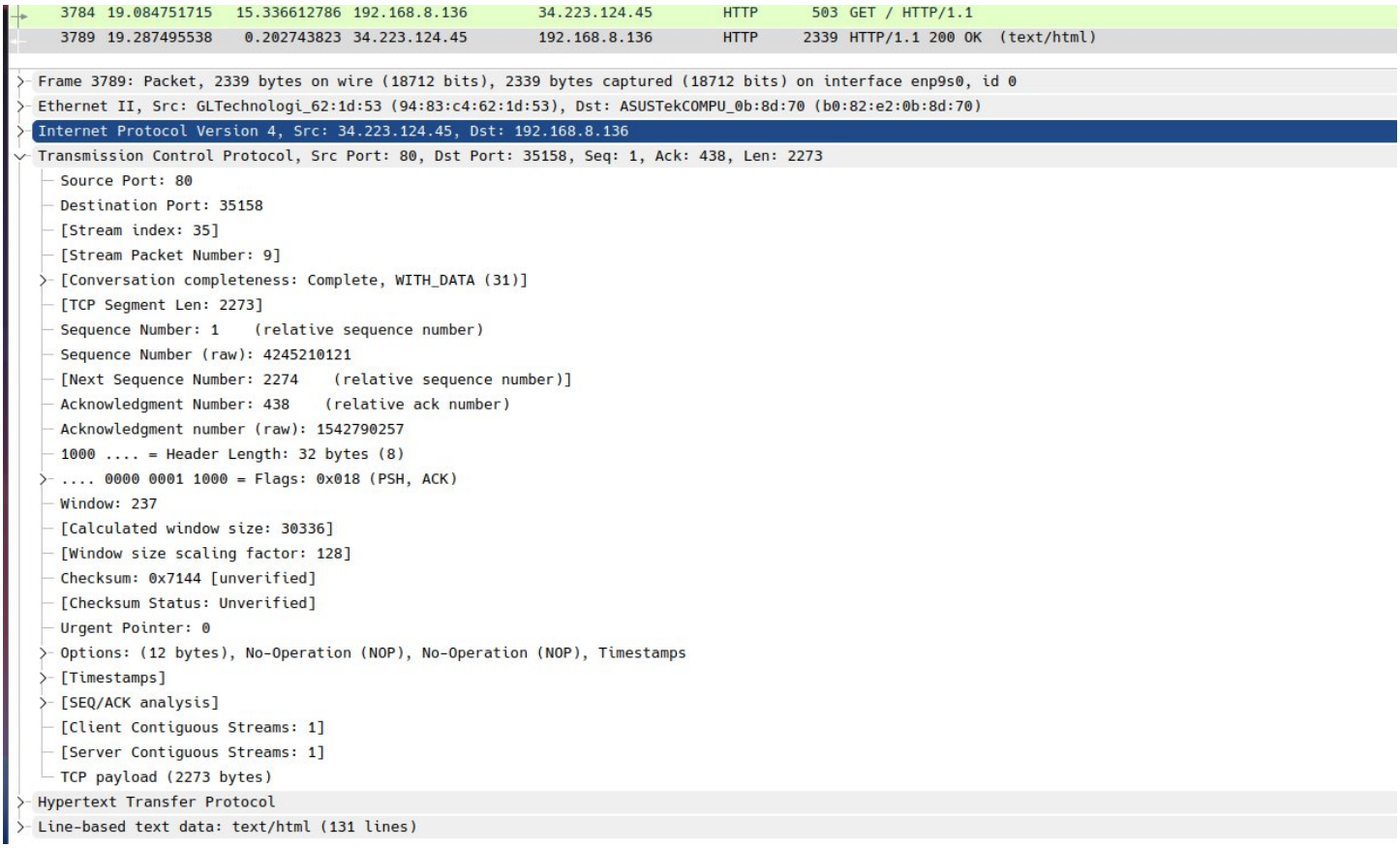
Below are the IPv4 information.





Below are the TCP related information.





a. What is the sequence of HTTP message exchange?

The client 192.168.8.136 sent GET / HTTP/1.1 to the server 34.223.124.45. And then, NeverSSL responded with a HTTP/1.1 200 OK.

b. Are we using persistence/ non-persistence connection?

The request contains: Connection: keep-alive and the response also contains: Connection: Upgrade, Keep-Alive and Keep-Alive: timeout=5, max=100.

This means that the TCP connection can remain open for up to five seconds and can handle a maximum of 100 requests instead of creating a new TCP connection for every request.

c. What are details you can find in HTTP GET message?

d. Check the packet details in the middle Wireshark packet details pane. Can you identify the details in Ethernet II / Internet Protocol Version 4 / Transmission Control Protocol / Hypertext Transfer Protocol frames?

Screenshots of this for both the request and the response has been attached above.

e. What is the HTTP version your browser used?

It's HTTP/1.1.

f. Identify the response message received from the server?

The NeverSSL server returned a HTTP/1.1 200 OK.

g. What is version of HTTP that the server is running?

The server responded using HTTP/1.1. The web server software it's using is Apache/2.4.66.

h. Can you identify the IP address of your computer?

192.168.8.136

i. Can you identify the IP address of the server?

34.223.124.45

j. What is the status code returned from the server to your browser?

200 OK

k. When was the HTML file that you are accessing last modified at the server?

Last-Modified: Wed, 29 Jun 2022 00:23:33 GMT

i. Repeat the analysis by accessing a different web page of your choice.

I first accessed http://www.bom.gov.au/ and it returned a 301 to it's HTTPS site. Images and explanations of this can be seen above.

8. What happens when you use "https"? Can you analyse the responses? Discuss your answer with your group members.

When HTTPS is used, both the request and the response will be encrypted with TLS. Wireshark can still show the IP addresses, port 443, TLS handshake, and the encrypted data that gets sent but it cannot normally display the GET request, status code, headers or web page content unless the browser's TLS session keys are provided.

The PCAP file that I used for this exercise can be downloaded here: https://drive.google.com/file/d/1H0wq0Hr-HlefHglHEn27XEQc0Dsppj4l/view?usp=sharing

References

[1] I. I. layer protocol, "Stack Overflow," Stack Overflow. Accessed: Aug. 01, 2026. [Online]. Available: https://stackoverflow.com/questions/19218596/is-icmp-a-transport-layer-protocol/19218648#19218648

[2] "Quic Http3," F5, Inc. Accessed: Aug. 01, 2026. [Online]. Available: https://www.f5.com/glossary/quic-http3

[3] J. Reschke, J. M. Snell, and M. Bishop, "RFC 10008: The HTTP QUERY Method," IETF Datatracker. Accessed: Aug. 01, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc10008

[4] H. shell script, "Stack Overflow," Stack Overflow. Accessed: Aug. 01, 2026. [Online]. Available: https://stackoverflow.com/questions/4497759/how-to-get-remote-file-size-from-a-shell-script/4497786#4497786

[5] "Cookie Attributes," 2025. Accessed: Aug. 01, 2026. [Online]. Available: https://privacysandbox.google.com/cookies/basics/cookie-attributes

[6] "How Google Uses Cookies – Privacy & Terms – Google." Accessed: Aug. 01, 2026. [Online]. Available: https://policies.google.com/technologies/cookies